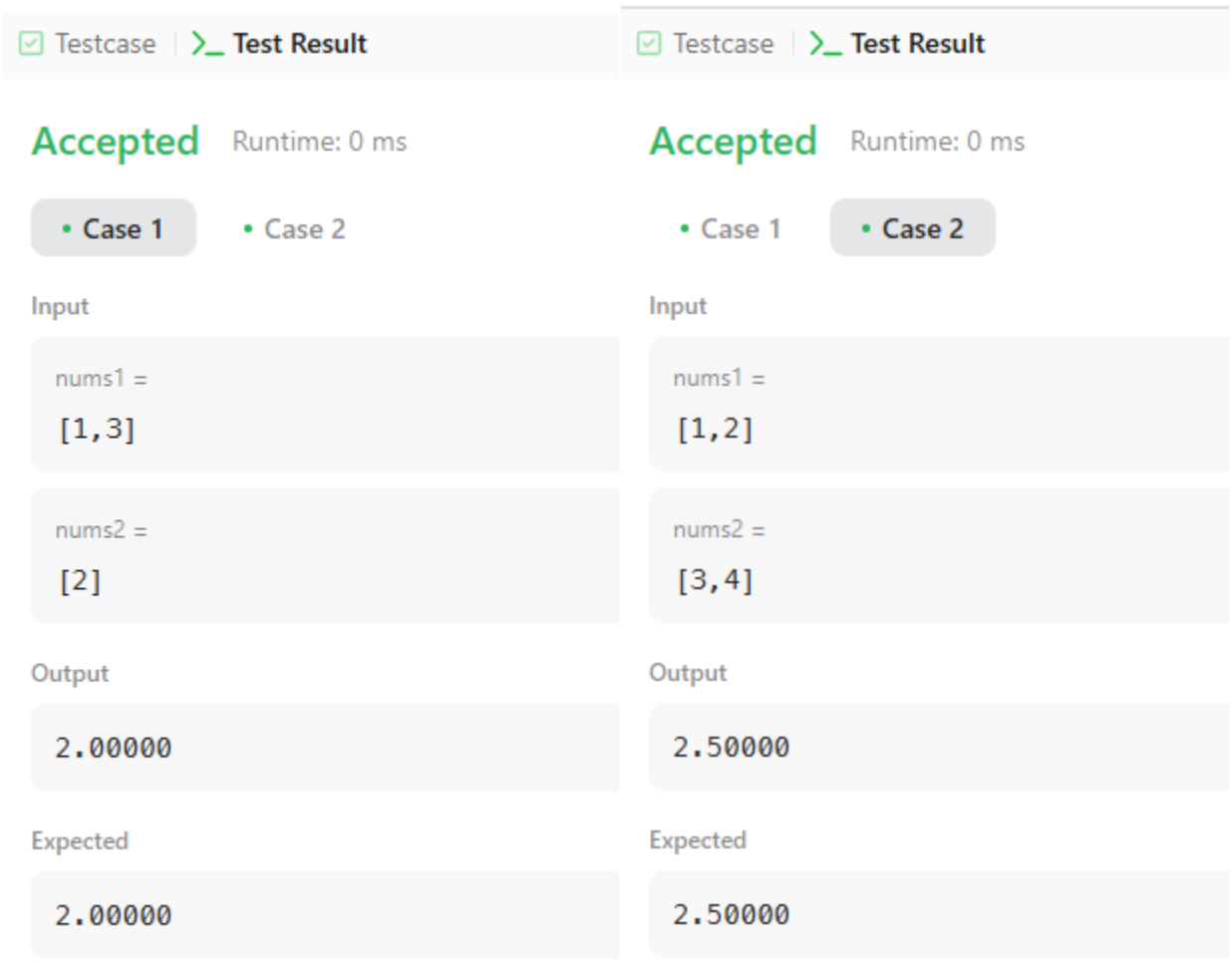
Median of Two Sorted Arrays

Hard

```
class Solution {
public:
  double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
    int m = nums1.size();
    int n = nums2.size();
    int totalLeft = (m + n + 1) / 2;
    int left = 0, right = m;
    while (left <= right) {
      int i = (left + right) / 2;
      int j = totalLeft - i;
      int nums1LeftMax  = (i == 0) ? INT_MIN : nums1[i - 1];
      int nums1RightMin = (i == m) ? INT_MAX : nums1[i];
      int nums2LeftMax  = (j == 0) ? INT_MIN : nums2[j - 1];
      int nums2RightMin = (j == n) ? INT_MAX : nums2[j];

      if (nums1LeftMax <= nums2RightMin && nums2LeftMax <= nums1RightMin) {

        if ((m + n) % 2 == 1) {
          return max(nums1LeftMax, nums2LeftMax);
        } else {
          return (max(nums1LeftMax, nums2LeftMax) + min(nums1RightMin,
nums2RightMin)) / 2.0;
        }
      } else if (nums1LeftMax > nums2RightMin) {
        right = i - 1;
      } else {
        left = i + 1;
      }
    }
    throw runtime_error("Input arrays are not sorted properly.");
  }
};
```



Approach:

1. We need to identify the size of both sorted arrays and then add them if count is even or odd.
2. Left half has the same number of elements as the right half (or one more if the total count is odd).
   All numbers on the left half ≤ all numbers on the right half.
3. **int totalLeft = (m + n + 1) / 2;**
   If odd total, biggest number on the left side.
   If even total, average of biggest number on the left and smallest number on the right.
4. Then, do binary search to correctly divide the halves.
   int nums1LeftMax = (i == 0) ? INT_MIN : nums1[i - 1]; //1-1
   int nums1RightMin = (i == m) ? INT_MAX : nums1[i];
   int nums2LeftMax = (j == 0) ? INT_MIN : nums2[j - 1];
   int nums2RightMin = (j == n) ? INT_MAX : nums2[j];